# 2-D Geometric Transformations

**Computer Graphics | Unit 2 | Classroom Notes**

Translation, Scaling, Rotation, Reflection, Shear • Matrix representation and homogeneous coordinates • Composite transforms • Transformations between coordinate systems

### How to use these notes

- **Convention:** column-vector style, **P′ = M · P**. Positive angle = counter-clockwise (CCW) in a y-up system. Some books use row vectors (P′ = P · M); see Section 6.
- **Colour code:** blue = definition, amber = important point, green = example, red = common mistake, purple = activity, teal = real-world link.
- **Prerequisites:** matrix multiplication, sin/cos of common angles (0°, 30°, 45°, 60°, 90°, 180°).
- These notes do not assume any particular university syllabus or textbook. Check your own syllabus for the exact depth required.

| Learning outcomes — after this unit you can… |
|---|
| 1. Write equations and matrices for translation, scaling, rotation, reflection and shear. |
| 2. Explain why homogeneous coordinates are needed and use 3×3 matrices. |
| 3. Build composite transformations (pivot rotation, fixed-point scaling) and explain why order matters. |
| 4. Convert coordinates between two coordinate systems (e.g. world to screen). |

## 1 Introduction: Why Transformations?

### DEFINITION: Geometric transformation

A **geometric transformation** changes the **position, size, orientation or shape** of an object by changing the coordinates of its points.

| Everyday situation | Transformation |
|---|---|
| Dragging a window or moving a game character | Translation |
| Pinch-to-zoom on a phone map | Scaling |
| Rotating a photo in an editing app | Rotation |
| “Flip horizontal” in a photo editor | Reflection |
| Italic text, slanted shadows | Shear |

A 2-D point is written as a column vector, and the new point is found by **P′ = M · P**, where **M** is the transformation matrix. An object (line, triangle, polygon) is transformed by transforming its **vertices** and redrawing the edges.

### IMPORTANT

All transformations in this unit map straight lines to straight lines, so transforming only the **end-points** is enough. This is why they are called **affine** transformations.

# 2 The Five Basic Transformations

Equations, matrices and behaviour

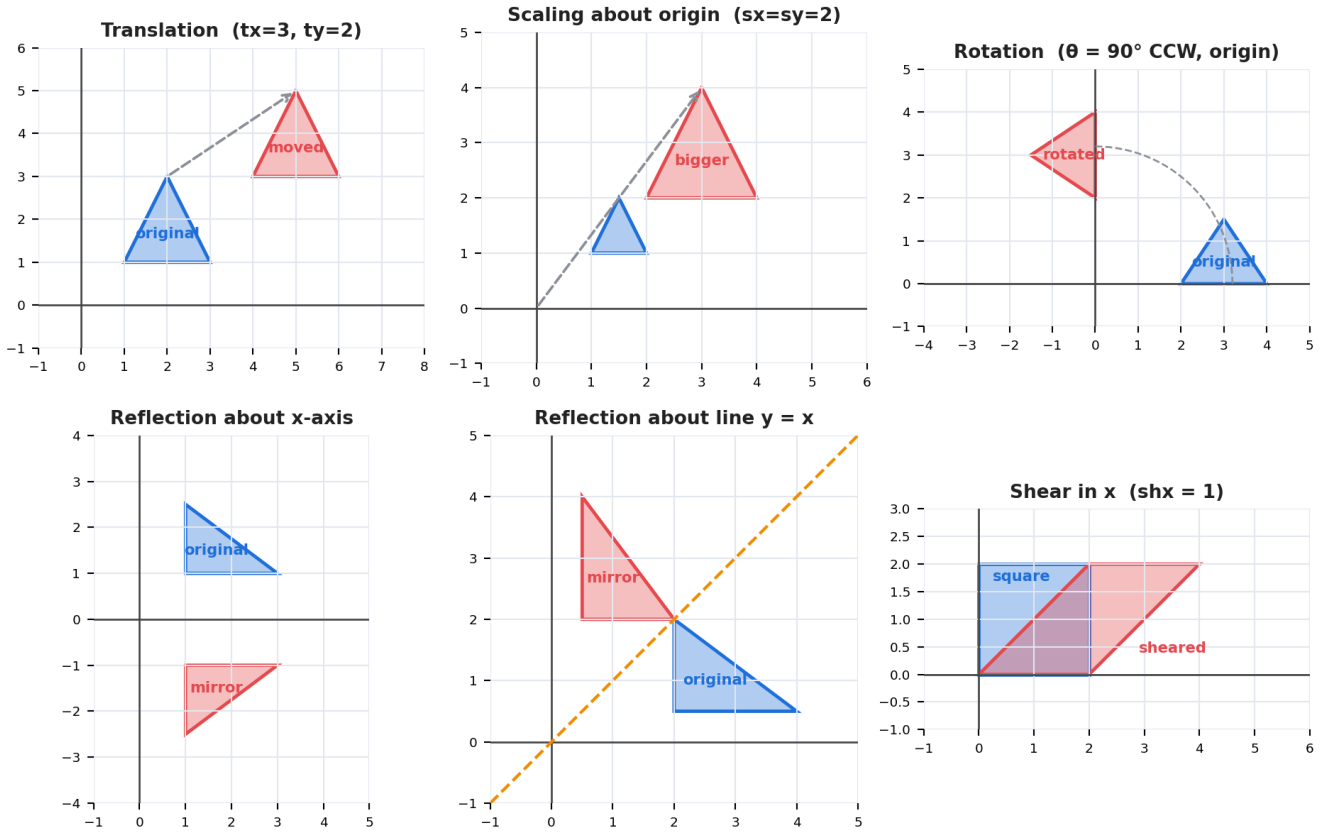



**Figure 1:** The five basic transformations (blue = original, red = result).

## 2.1 Translation

**DEFINITION**

**Translation** moves every point of an object by the same distance along a straight path. (tx, ty) is the **translation vector**.

```
x' = x + tx
y' = y + ty
```

- Preserves shape, size and orientation (a **rigid-body** transformation).
- In a 2×2 matrix form translation is an **addition**, not a multiplication. That is the reason we need homogeneous coordinates (Section 3).

**REAL-WORLD**

Moving a character in a game by (5, 0) pixels per frame; dragging an icon on screen.

## 2.2 Scaling

**DEFINITION**

**Scaling** changes the size of an object using scale factors **sx** and **sy**.

| x' = sx·x , y' = sy·y | Matrix S(sx, sy) |
|---|---|
| $\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} sx \cdot x \\ sy \cdot y \end{bmatrix}$ | $\begin{bmatrix} sx & 0 \\ 0 & sy \end{bmatrix}$ |

| Case | Meaning |
|---|---|
| sx = sy > 1 | Uniform enlargement |
| sx = sy < 1 (but > 0) | Uniform shrinking |
| sx ≠ sy | **Differential (non-uniform) scaling**: shape is stretched |
| sx or sy negative | Scaling combined with reflection |

**IMPORTANT**

Standard scaling is about the **origin**, so the object also moves away from (or towards) the origin. To scale about the object's own centre, use a **fixed point** (Section 4).

## 2.3 Rotation

**DEFINITION**

**Rotation** turns an object through angle **θ** about a pivot point. By convention **positive θ is counter-clockwise** in a y-up (right-handed) system.

**Derivation:** let P = (r cos φ, r sin φ). After rotating by θ, the point is (r cos(φ+θ), r sin(φ+θ)). Using the angle-addition formulas:

```
x' = x cosθ − y sinθ
y' = x sinθ + y cosθ
```

| Rotation R(θ) | Inverse R(−θ) = Rᵀ |
|---|---|
| $\begin{vmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{vmatrix}$ | $\begin{vmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{vmatrix}$ |

- Preserves lengths and angles. Determinant = 1. The inverse equals the transpose.

**IMPORTANT: screen y-axis**

On most screens the y-axis points **down**, so a “positive” rotation looks **clockwise** there. Always check the coordinate convention.

## 2.4 Reflection

**DEFINITION**

**Reflection** produces a **mirror image** of an object about a line (axis) or a point.

| Reflection about | (x′, y′) | Matrix (2×2) |
|---|---|---|
| x-axis | (x, −y) | [1 0 ; 0 −1] |
| y-axis | (−x, y) | [−1 0 ; 0 1] |
| Origin | (−x, −y) | [−1 0 ; 0 −1] |
| Line y = x | (y, x) | [0 1 ; 1 0] |
| Line y = −x | (−y, −x) | [0 −1 ; −1 0] |

- Reflection **reverses orientation** (determinant = −1): clockwise vertex order becomes anticlockwise.
- Reflection about the origin is the same as a **180° rotation** in 2-D.

## 2.5 Shear

**DEFINITION**

**Shear** slides the layers of an object parallel to an axis, distorting its shape, like pushing the top of a deck of cards sideways.

| x-shear: x' = x + shx·y , y' = y | | y-shear: x' = x , y' = y + shy·x | |
|---|---|---|---|
| 1 | shx | 1 | 0 |
| 0 | 1 | shy | 1 |

- Shear changes angles but **preserves area** (determinant = 1).
- With a reference line y = yref: x′ = x + shx·(y − yref).
- Uses: italic fonts, slanted shadows, wind-blown grass and trees in games.

# 3 Matrix Representation and Homogeneous Coordinates

**THE PROBLEM**

Rotation and scaling are **multiplications** (P′ = M·P) but translation is an **addition** (P′ = P + T). So we cannot combine translation with the others in one 2×2 matrix.

**DEFINITION: Homogeneous coordinates**

A 2-D point (x, y) is represented by a triple **(xh, yh, h)** where **x = xh/h**, **y = yh/h** and h ≠ 0. We normally choose **h = 1**, so the point becomes **(x, y, 1)**. Now every transformation is a **3×3 matrix multiplication**.

| Translation T(tx,ty) | | | Scaling S(sx,sy) | | | Rotation R(θ) | | |
|---|---|---|---|---|---|---|---|---|
| 1 | 0 | tx | sx | 0 | 0 | cosθ | −sinθ | 0 |
| 0 | 1 | ty | 0 | sy | 0 | sinθ | cosθ | 0 |
| 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 1 |

| x-Shear | | | y-Shear | | | Reflection (x-axis) | | |
|---|---|---|---|---|---|---|---|---|
| 1 | shx | 0 | 1 | 0 | 0 | 1 | 0 | 0 |
| 0 | 1 | 0 | shy | 1 | 0 | 0 | −1 | 0 |
| 0 | 0 | 1 | 0 | 0 | 1 | 0 | 0 | 1 |

**INVERSES**

- $T^{-1} = T(-tx, -ty)$
- $S^{-1} = S(1/sx, 1/sy)$
- $R^{-1} = R(-\theta) = R^{T}$

| Feature | Ordinary 2×2 | Homogeneous 3×3 |
|---|---|---|
| Translation as matrix multiplication | No (needs addition) | Yes |
| Combining many transformations | Difficult | Multiply matrices into one |
| Uniform treatment of all transformations | No | Yes |
| Storage / computation | Less | Slightly more |

**KEY IDEA**

Homogeneous coordinates let us write translation, scaling, rotation, reflection and shear in one common form: **P′ = M · P**.

# 4 Composite (Concatenated) Transformations

**DEFINITION**

A **composite transformation** is a sequence of basic transformations combined by multiplying their matrices into a **single matrix**.

```
Apply M1 first, then M2, then M3 (column vectors):

      P' = M3 · M2 · M1 · P        <- M1 is applied FIRST (rightmost)
```

**IMPORTANT**

Matrix multiplication is generally **not commutative**, so the **order matters**.

## 4.1 Which pairs commute?

| Pair of transformations | Order doesn't matter? (commutative) | Reason |
|---|---|---|
| Translation, Translation | Yes | T(a)·T(b) = T(a+b) |
| Scaling, Scaling | Yes | Factors multiply |
| Rotation, Rotation (same pivot) | Yes | Angles add |
| Uniform scaling, Rotation | Yes | sx = sy, so the matrices commute |
| Translation, Rotation | **No** | Rotation is about the origin |
| Translation, Scaling | **No** | Scaling is about the origin |
| Rotation, Non-uniform scaling | **No** | Stretch direction changes after rotation |

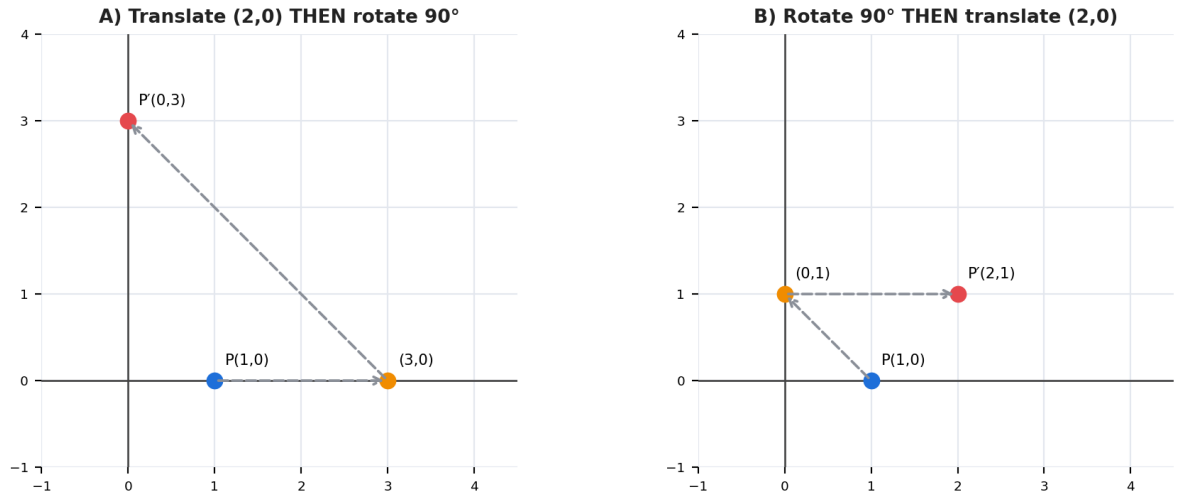



**Figure 2:** Same point, same two operations, different order gives different results: (0, 3) versus (2, 1).

## 4.2 Rotation about an arbitrary pivot (xr, yr)

1. Translate the pivot to the origin: **T(−xr, −yr)**
2. Rotate about the origin: **R(θ)**

3. Translate back: **T(xr, yr)**

```
M = T(xr,yr) · R(θ) · T(−xr,−yr)
```

### 4.3 Scaling about a fixed point (xf, yf)

```
M = T(xf,yf) · S(sx,sy) · T(−xf,−yf)
```

### 4.4 Reflection about an arbitrary line (general method)

Translate the line so it passes through the origin → rotate so the line lies on an axis → reflect about that axis → undo the rotation → undo the translation.

## 5 Practical (Solved) Examples

**Example 1: Translation**

Triangle A(2,2), B(6,2), C(4,6) translated by (3, −1).
**A′ = (5,1), B′ = (9,1), C′ = (7,5).**

**Example 2: Rotation about the origin**

Rotate P(4,3) by 90° CCW. cos 90° = 0, sin 90° = 1.
x′ = 4(0) − 3(1) = −3, y′ = 4(1) + 3(0) = 4. **P′ = (−3, 4).**

**Example 3: Rotation about a pivot**

Rotate P(4,2) by 90° CCW about (2,2). M = T(2,2)·R(90°)·T(−2,−2) gives x′ = −y + 4, y′ = x.
**P′ = (2, 4).** Check: translate to (2,0) → rotate to (0,2) → translate back to (2,4).

**Example 4: Scaling about the centre**

Square A(2,2), B(4,2), C(4,4), D(2,4), centre (3,3), scale factor 2. M = T(3,3)·S(2,2)·T(−3,−3) gives x′ = 2x − 3, y′ = 2y − 3.
**A′(1,1), B′(5,1), C′(5,5), D′(1,5).** The square doubles and stays centred. (Scaling about the origin would give (4,4), (8,4), (8,8), (4,8): it moves away.)

**Example 5: Shear**

Unit square (0,0), (1,0), (1,1), (0,1) with x-shear shx = 2: x′ = x + 2y.
**New vertices: (0,0), (1,0), (3,1), (2,1).**

**Example 6: Reflection**

Point (3,5) reflected about y = x gives **(5,3)**.

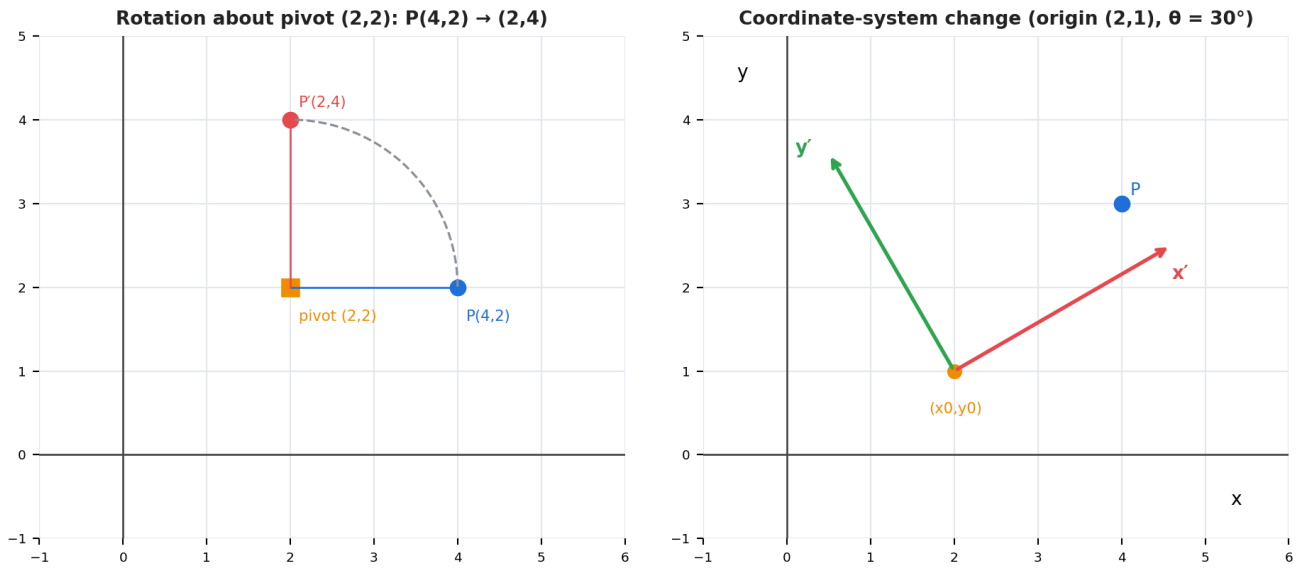



**Figure 3:** Left: pivot rotation (Example 3). Right: a second coordinate system with origin (x0, y0) rotated by θ (Section 7).

## 6 Properties Summary and Comparisons

| Transformation | Preserves length | Preserves angles | Preserves area | Determinant |
|---|---|---|---|---|
| Translation | Yes | Yes | Yes | 3×3 matrix: 1 |
| Rotation | Yes | Yes | Yes | 1 |
| Reflection | Yes | Yes | Yes | −1 |
| Uniform scaling (s) | No | Yes | No | $s^2$ |
| Non-uniform scaling | No | No | No | sx·sy |
| Shear | No | No | Yes | 1 |

**IMPORTANT**

**Rigid-body** transformations (translation, rotation, reflection) keep distances. All transformations here are **affine**: they preserve straight lines and parallelism.

| | Column-vector style | Row-vector style |
|---|---|---|
| Form | $P' = M \cdot P$ | $P' = P \cdot M$ |
| Matrices | As shown in these notes | Transposed (tx, ty in the bottom row) |
| Composite order | Right to left (rightmost first) | Left to right (leftmost first) |

**COMMON MISTAKE**

Textbooks and graphics APIs differ in convention. Always **state the one you use** in an exam answer.

## 7 Transformations Between Coordinate Systems

Sometimes the object stays fixed and we describe the **same point in a different coordinate system**. Examples: modelling (local) → world → screen (device) coordinates.

**Setting:** the new system has its origin at (x0, y0) of the old system, and its axes are rotated by angle θ.

1. Translate so the new origin coincides with the old origin: **T(−x0, −y0)**

2. Rotate to align the axes: **R(−θ)**

```
M(old -> new) = R(-θ) · T(-x0, -y0)
M^-1 (new -> old) = T(x0, y0) · R(θ)
```

**KEY IDEA**

Changing the coordinate system is equivalent to applying the **inverse** transformation to the object.

**Example: world (y-up) to screen (y-down)**

Screen height = H. For a point (x, y): **y_screen = H − y**, x_screen = x. This is a reflection about the x-axis followed by a translation by H:

**T(0,H) · Reflection(x-axis)**

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & -1 & H \\ 0 & 0 & 1 \end{bmatrix}$$

**Numerical example**

New origin at (2,1), axes rotated 90° CCW. Find the new coordinates of P(5,3).
Translate: (5−2, 3−1) = (3, 2). Apply R(−90°): x′ = x cos(−90°) − y sin(−90°) = 0 + 2 = 2, y′ = x sin(−90°) + y cos(−90°) = −3. **P in the new system = (2, −3).**
Check: new x′-axis points along old +y, new y′-axis points along old −x. Vector (3,2) has component 2 along old y (so x′ = 2) and −3 along old x (so y′ = −3). ✓

**REAL-WORLD**

A game character (local coordinates) is placed in the world, then the camera converts the world to screen pixels. Each step is one matrix, and all are multiplied into one.

## 8 Common Student Mistakes

**Avoid these 12 mistakes**

1. Applying matrices in the **wrong order**, e.g. writing M1·M2·P when M1 should be applied first.
2. Trying to do translation with a **2×2 matrix**.
3. Rotating about a pivot **without** translating the pivot to the origin first.
4. Forgetting the final **translate-back** step.
5. Mixing **degrees and radians** in a calculator or in code.
6. Wrong sign of angle (clockwise vs anticlockwise).
7. Forgetting that scaling about the origin also **moves** the object.
8. Putting tx, ty in the bottom row of a column-vector matrix (that is row-vector style).
9. Not using h = 1, or forgetting to divide by h at the end.
10. Putting sin and cos in the wrong positions of the rotation matrix.
11. Confusing reflection about the **origin** with reflection about an **axis**.
12. Assuming the inverse of a scale or shear is the same matrix (inverse scale = 1/s; inverse shear = −sh).

## 9 5-Minute Classroom Activity: “Be the Transformation”

**Task**

1. Draw triangle **P(1,1), Q(4,1), R(1,3)** on graph paper.
2. Work in pairs. **Student A**: rotate 90° CCW about the origin, **then** translate by (2, 0). **Student B**: translate by (2, 0), **then** rotate 90° CCW about the origin. (3 min)
3. Compare results and discuss: *why are the answers different?* (2 min)

| | Step 1 | Final result |
|---|---|---|
| Student A (rotate → translate) | (−1,1), (−1,4), (−3,1) | **P′(1,1), Q′(1,4), R′(−1,1)** |
| Student B (translate → rotate) | (3,1), (6,1), (3,3) | **P′(−1,3), Q′(−1,6), R′(−3,3)** |

**Learning outcome**

The **order** of composite transformations matters.

## 10 Interview Questions

| # | Question | Key answer |
|---|---|---|
| 1 | What are homogeneous coordinates and why are they used? | Represent (x,y) as (x,y,1); lets translation be a matrix multiplication so all transformations combine into one matrix. |
| 2 | Why a 3×3 matrix for 2-D transformations? | One extra dimension makes translation linear in the matrix form. |
| 3 | Is matrix multiplication commutative? Example from graphics? | No. Translate then rotate differs from rotate then translate. |
| 4 | How do you rotate an object about its own centre? | T(centre)·R(θ)·T(−centre). |
| 5 | Inverse of a rotation matrix? | R(−θ), which equals its transpose. |
| 6 | Reflection about the origin vs rotation by 180°? | Same in 2-D. A reflection about an axis is different: it flips orientation (det = −1). |
| 7 | What is an affine transformation? | Preserves straight lines and parallelism (a linear map plus a translation). |
| 8 | Which transformations are rigid-body? | Translation, rotation (and reflection if mirror images are allowed). |
| 9 | What does the determinant tell you? | Area scale factor. A negative sign means orientation is reversed. |
| 10 | How do you convert world to screen coordinates? | Flip y (y_screen = H − y) and scale/translate to the viewport. |
| 11 | Real use of shear? | Italic text, slanted shadows, animation effects. |
| 12 | Why does order matter in game engines? | Different order gives different position and orientation of an object. |

## 11 University Examination Questions

### Short answer (2–3 marks)

1. Define translation and write its homogeneous matrix.
2. What is uniform scaling?
3. Write the 2-D rotation matrix and its inverse.
4. What are homogeneous coordinates?
5. Write the matrix for reflection about the y-axis.
6. Define shear. Give the x-shear matrix.
7. What is a composite transformation?

### Medium answer (5–6 marks)

1. Derive the 2-D rotation equations about the origin.

2. Explain why homogeneous coordinates are needed. Give all basic 3×3 matrices.
3. Find the new coordinates of triangle A(1,1), B(3,1), C(2,4) after scaling by (2,3) about the origin. *[Answer: A′(2,3), B′(6,3), C′(4,12)]*
4. Explain reflection about the x-axis, y-axis, origin and the line y = x with matrices.
5. Show that two successive rotations are additive: R(θ1)·R(θ2) = R(θ1+θ2).
6. Rotate P(3,1) by 90° CCW about (1,1). *[Answer: P′(1,3)]*

## 12 Five Long-Answer Questions

**DEFINITION**

1. Explain all five basic 2-D geometric transformations with equations, matrices, diagrams and real-world examples.
2. What are homogeneous coordinates? Derive the 3×3 matrices for translation, scaling and rotation and explain the advantages over ordinary coordinates.
3. Derive the composite matrix for rotation about an arbitrary pivot (xr, yr). Apply it to rotate triangle A(2,3), B(5,3), C(4,6) by 90° CCW about (2,2) and give the new vertices.
4. Explain composite transformations. Discuss with examples which pairs of transformations commute and which do not. Derive the matrix for scaling about a fixed point.
5. Explain transformation between two coordinate systems. Derive the matrix to convert coordinates from a system with origin (x0, y0) and axes rotated by θ, and explain world-to-screen conversion with an example.

**Answer key for Q3**

M = T(2,2)·R(90°)·T(−2,−2) gives x′ = −y + 4, y′ = x. **A′(1,2), B′(1,5), C′(−2,4).**

## 13 Short Recap

**Remember**

- A **transformation** changes position, size, orientation or shape by changing coordinates.
- Five basics: **translation, scaling, rotation, reflection, shear**.
- **Homogeneous coordinates** (x, y, 1) turn all of them into 3×3 matrix multiplications.
- **Composite = matrix product**. Order matters. Rightmost matrix is applied first.
- Pivot rotation = **T(pivot)·R·T(−pivot)**. Fixed-point scaling works the same way.
- Coordinate-system change = **R(−θ)·T(−x0, −y0)**.
- Rotation, reflection, translation keep lengths. Shear keeps area. Uniform scaling keeps angles.